

```

for(int i = 0; ; i++) {
    double term = 1.0 / (i*2+1);
    if(i % 2 == 0) sum += term;
    else sum -= term;
    if(term < 1e-6) break;
}
printf("%.6f\n", sum);
return 0;
}

```

提示 2-14: do-while 循环的格式为“do { 循环体 }while(条件);”，其中循环体至少执行一次，每次执行完循环体后判断条件，当条件满足时继续循环。

2.3 循环的代价

例题 2-4 阶乘之和

输入 n ，计算 $S = 1! + 2! + 3! + \cdots + n!$ 的末 6 位（不含前导 0）。 $n \leq 10^6$ ， $n!$ 表示前 n 个正整数之积。

样例输入：

10

样例输出：

37913

【分析】

这个任务并不难，引入累加变量 S 之后，核心算法只有“for(int i = 1; i <= n; i++) S += i!”。不过，C 语言并没有阶乘运算符，所以这句话只是伪代码，而不是真正的代码。事实上，还需要一次循环来计算 $i!$ ，即“for(int j = 1; j <= i; j++) factorial *= j;”。代码如下：

程序 2-7 阶乘之和 (1)

```

#include<stdio.h>
int main()
{
    int n, S = 0;
    scanf("%d", &n);
    for(int i = 1; i <= n; i++)
    {
        int factorial = 1;
        for(int j = 1; j <= i; j++)
            factorial *= j;
    }
}

```

```

    S += factorial;
}
printf("%d\n", S % 1000000);
return 0;
}

```

注意累乘器 `factorial`（英文“阶乘”的意思）定义在循环里面。换句话说，每执行一次循环体，都要重新声明一次 `factorial`，并初始化为 1（想一想，为什么不是 0）。因为只要末 6 位，所以输出时对 10^6 取模。

提示 2-15：在循环体开始处定义的变量，每次执行循环体会重新声明并初始化。

有了刚才的经验，下面来测试一下这个程序： $n=100$ 时，输出 -961703。直觉告诉我们：乘法又溢出了。这个直觉很容易通过“输出中间变量”法得到验证，但若解决这个问题，还需要一点数学知识。

提示 2-16：要计算只包含加法、减法和乘法的整数表达式除以正整数 n 的余数，可以在每步计算之后对 n 取余，结果不变。

在修正这个错误之前，还可以进行更多测试：当 $n=10^6$ 时输出什么？更会溢出不是吗？但是重点不在这里。事实上，它的速度太慢！下面把程序改成“每步取模”的形式，然后加一个“计时器”，看看究竟有多慢。

程序 2-8 阶乘之和（2）

```

#include<stdio.h>
#include<time.h>
int main()
{
    const int MOD = 1000000;
    int n, S = 0;
    scanf("%d", &n);
    for(int i = 1; i <= n; i++)
    {
        int factorial = 1;
        for(int j = 1; j <= i; j++)
            factorial = (factorial * j % MOD);
        S = (S + factorial) % MOD;
    }
    printf("%d\n", S);
    printf("Time used = %.2f\n", (double)clock() / CLOCKS_PER_SEC);
    return 0;
}

```

上面的程序再次使用到了常量定义，好处是可以在程序中使用代号 `MOD` 而不是常数